

# Plan first

Applying the NAF Framework to real life projects

AUTOCOONS

THE NETWORK AUTOMATION CONFERENCE

A long time ago..

# NAF Framework Inputs

## Request

- Bi-Directional Sync LDAP <-> Infrastructure
  - Add/Update/Delete
- Customer Specific Business Logic

## Idea

- Event-Driven Architecture, Auto-Remediation
- Non-Persistent State
- No UI

## Outcome

- Robust Sync
- Manual changes to Infra doesn't break system
- Survivability for Reboots, Container Wipe

Well...

- ❑ Customer was happy, but...
- ❑ Follow-up project disclosed issues:
  - ❑ Complex codebase
  - ❑ User manual but no design documentation
  - ❑ Internal mechanisms required reverse engineering

No success by our definition

## What went wrong?

- ❑ No design process
- ❑ Requirements where met but execution ignored future expansion
- ❑ Project knowledge stored in developer brains

## About Me

- ❖ IT since 2001
- ❖ First automation project in 2003
- ❖ Path Engineer to Solutions Architect
- ❖ Everything is more fun in a great team
- ❖ Cologne based
- ❖ Sailing is my non-IT passion

# Mission

PRESENTS

# DESIGN-FIRST, TRUST-ALWAYS: SUMMARIZING WHAT WE LEARNED AT #AUTOCON3

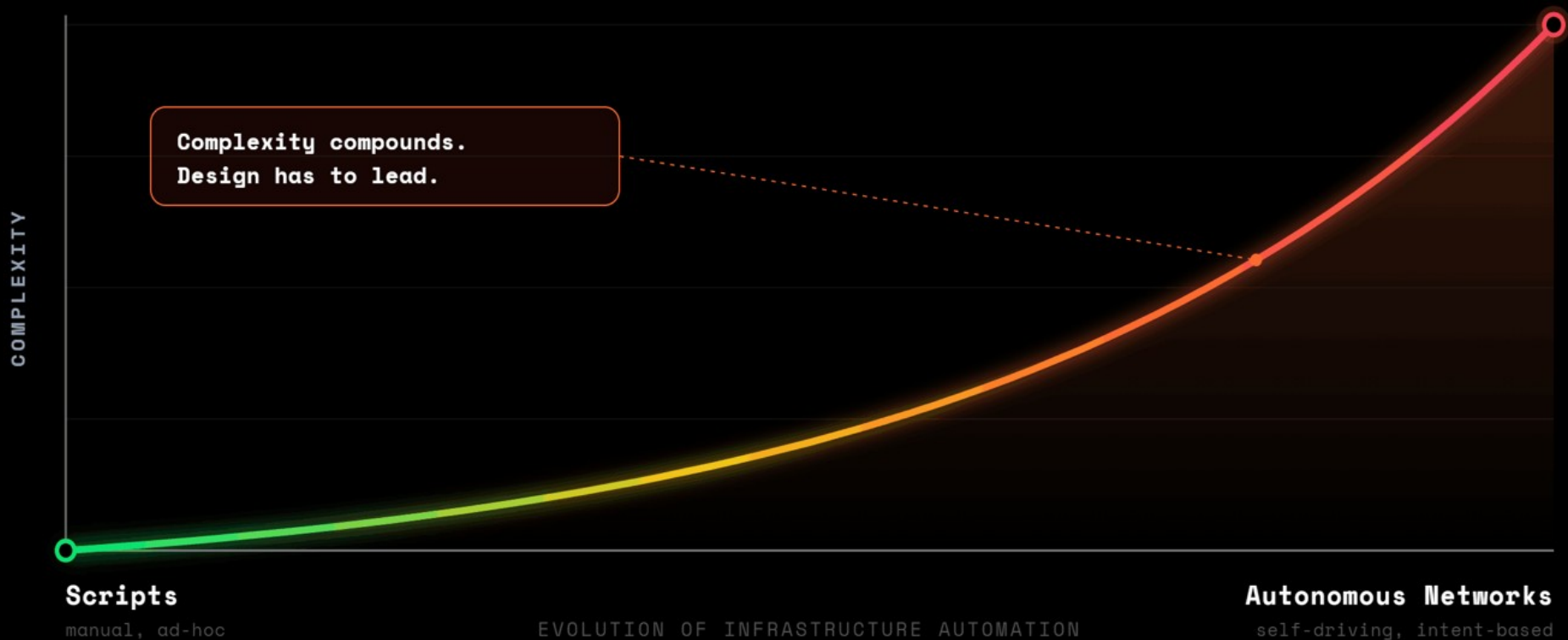
NETWORKAUTOMATION.FORUM



# Motivations Behind Automation Design

- ❑ Business Goals Alignment
- ❑ Prevention of Gaps and Silos
- ❑ Clear Responsibility Definitions
- ❑ Extensibility, Maintainability, Scalability
- ❑ Vendor-/Tool-Agnosticism
- ❑ Avoiding Knowledge Silos
- ❑ Regulation
- ❑ Documentation

# The Complexity Challenge



# NAF Framework taking the stage



# NAF Framework Inputs

## Building Blocks

- Mapping to system design structure
- Functional Requirements
- Tool Selection

## Responsibilities

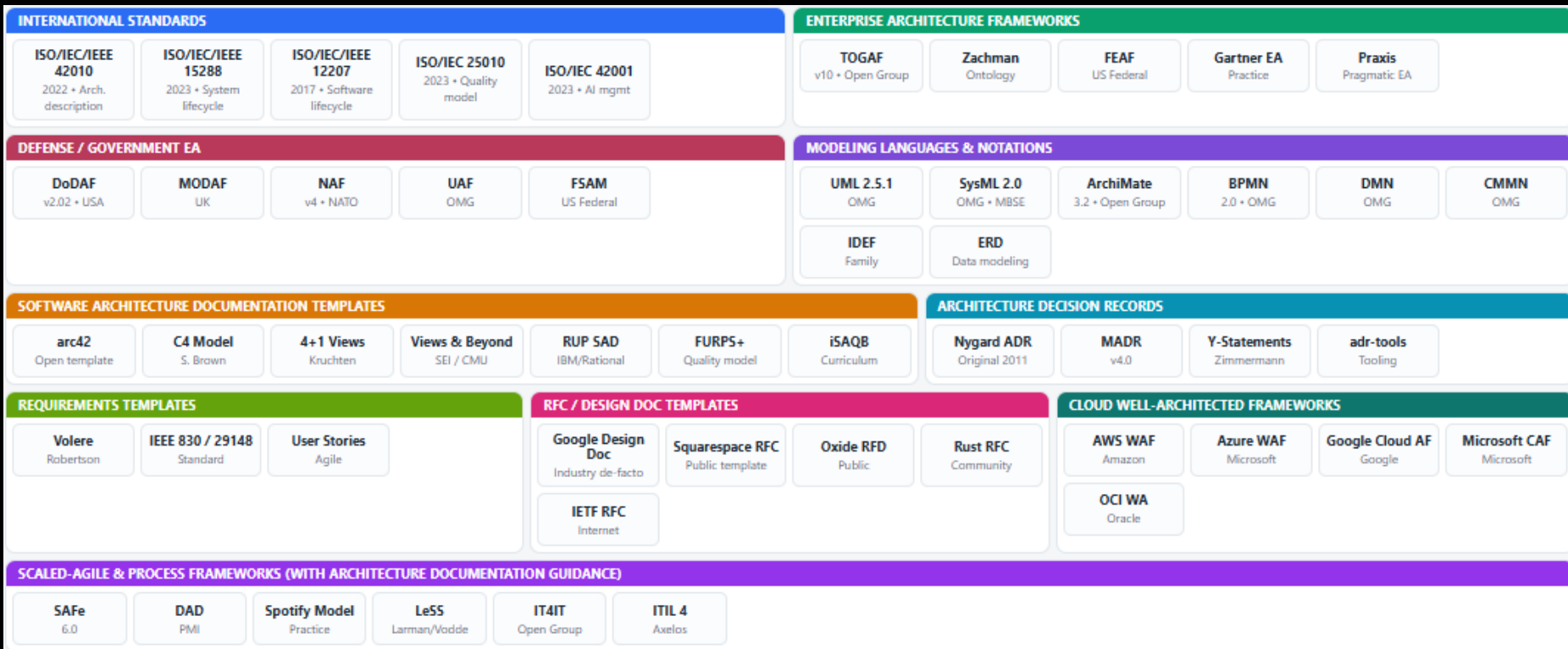
- Functional Requirements
- Clarification of Responsibilities
- Translation to Interfaces/API Design

## Brand Guidelines

- Common visual language
- Color Coding of system components

# Gear

# Eco System of Standards



# Personal Gear - arc42

## **arc42 - Documentation Template**

- Developed by Gernot Starke and Peter Hruschka
- Focus on communication and understandability
- Optimized for Software Architecture, works for Systems Design as well

# Scenario



# Project Use Case 1

## Day-0 Deployment

- Configuration of “fresh” devices
- Zero-Touch
- Location: Logistics Hub
- Users: Logistics employees
- IT only informatively involved

# Project Use Case 2

## Day-2 Configuration

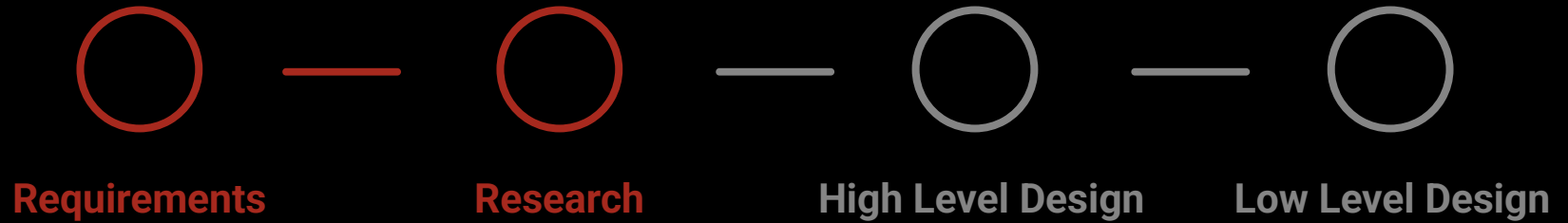
- Device management in operation
- Location: Remote sites nationwide
- Users: IT Networking Team

## Other Requirements

- Compliance: BSI IT-Grundschutz, NIS2
- On-Premise only
- Loss of location connectivity must not disrupt local services.

# Strategy

# Design Process



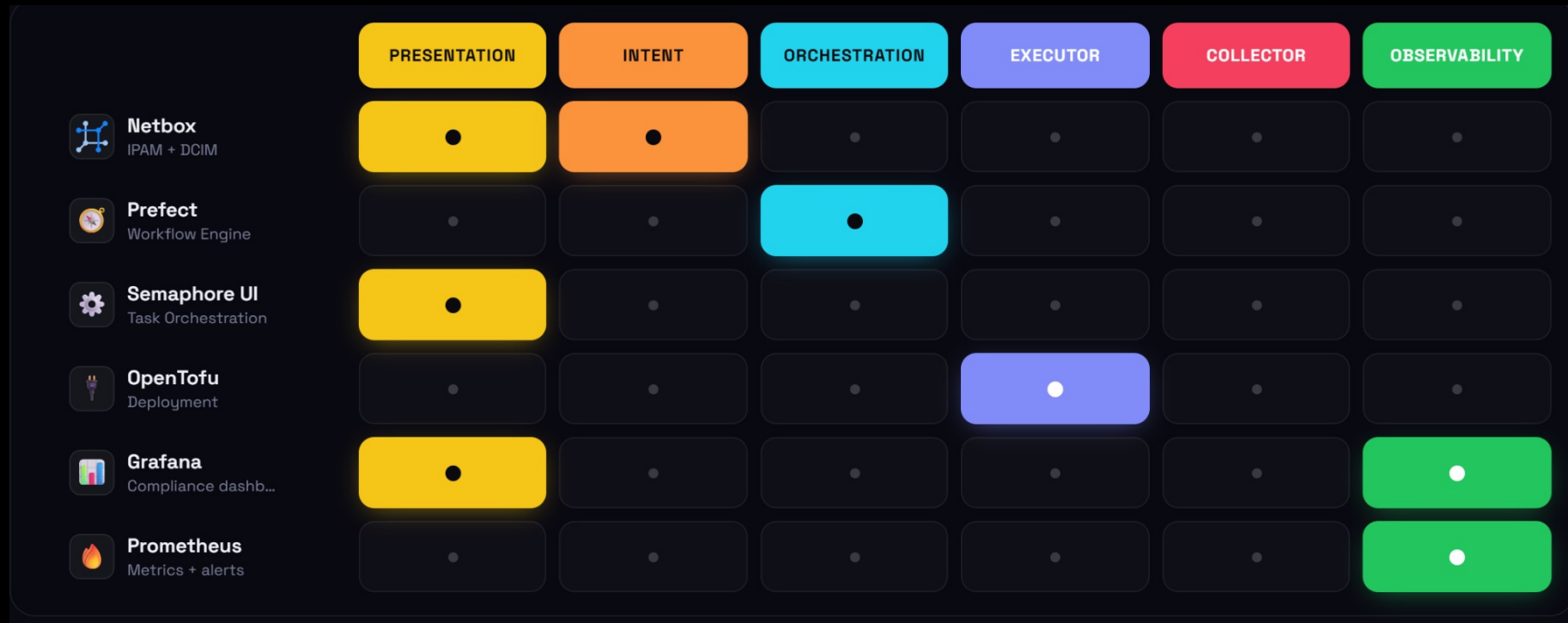
## Early-On Decisions

- Simplified Front-End for logistics employees
- Datacenter team has established automation practice, alignment is preferred
- Multi-Vendor strategy

# Architecture (First Draft)

- NAF Framework
- Workflow Engine
- YANG
- Observability yet undecided
- Container based deployments
- Others:
  - Minimal persistence, customer tooling,
  - BSI compliance as ADR\* quality gate

# Stack





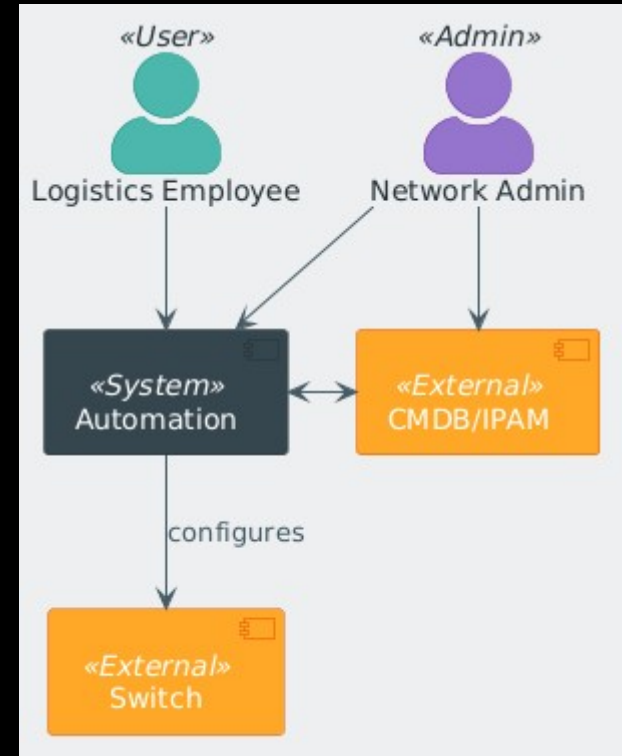
# Toolbox

- arc42
- ADR template
- PlantUML
- Mermaid
- Confluence

# Artifacts

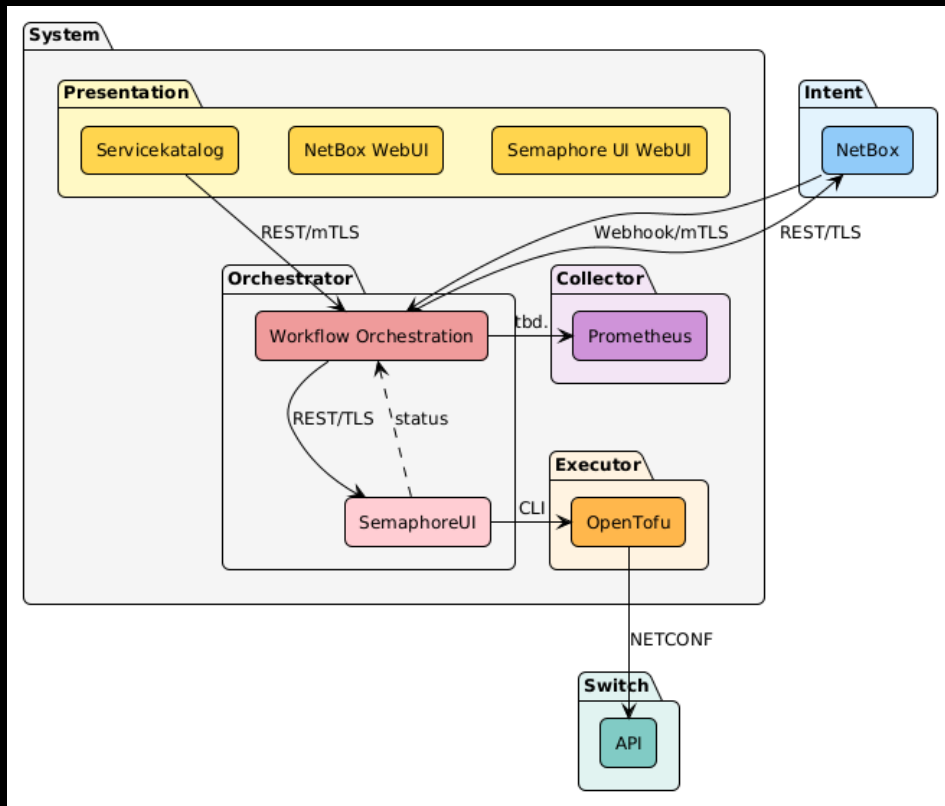
# Business Context

- Use Case Overview
- Very High Level
- Relations:
  - user : systems
  - systems : systems
- “System:Automation”:  
The new automation platform

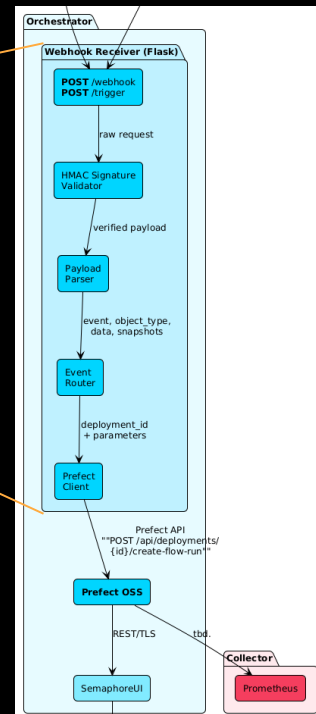
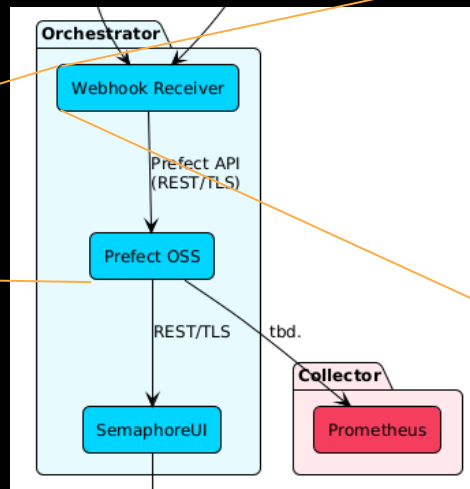
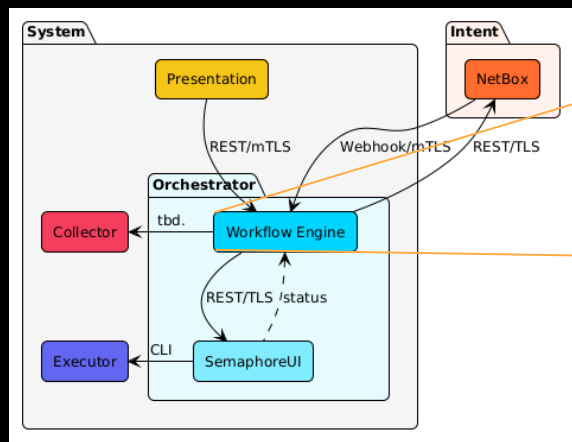


# Technical Context

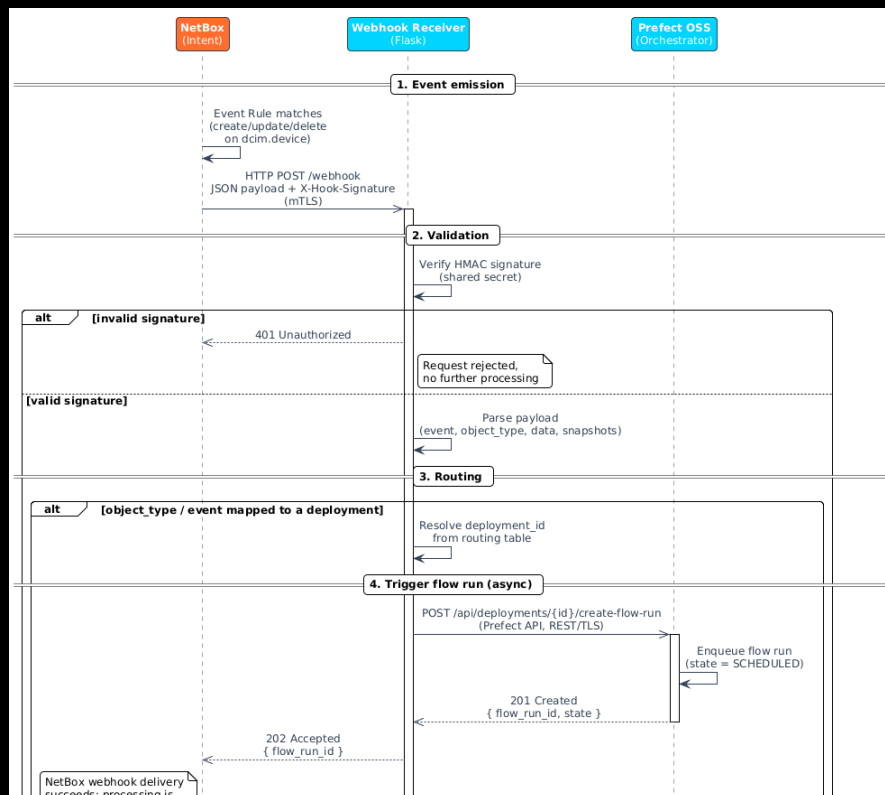
```
23 ' — Intent —————
24 package "Intent" as intent #E3F2FD {
25     component "NetBox" as cmdb #90CAF9
26 }
27
28 ' — System —————
29 package "System" as system #F5F5F5 {
30
31     package "Presentation" as presentation #FFF9C4 {
32         component "Servicecatalog" as servicecatalog #FFD54F
33         component "NetBox WebUI" as netbox_ui #FFD54F
34         component "Semaphore UI WebUI" as orchestrator_ui #FFD54F
35     }
36
37     package "Orchestrator" as orchestrator #F5F5F5 {
38         component "Workflow Orchestration" as workflow_engine #EF9A9A
39         component "SemaphoreUI" as orchestrator_backend #FFCDD2
40     }
41
42     package "Collector" as collector #F3E5F5 {
43         component "Prometheus" as prometheus #CE93D8
44     }
45
46     package "Executor" as executor #FFF3E0 {
47         component "OpenTofu" as opentofu #FFB74D
48     }
49 }
50
51 ' — Device —————
52 package "Switch" as device #E0F2F1 {
53     component "API" as device_api #80CBC4
54 }
55 }
```



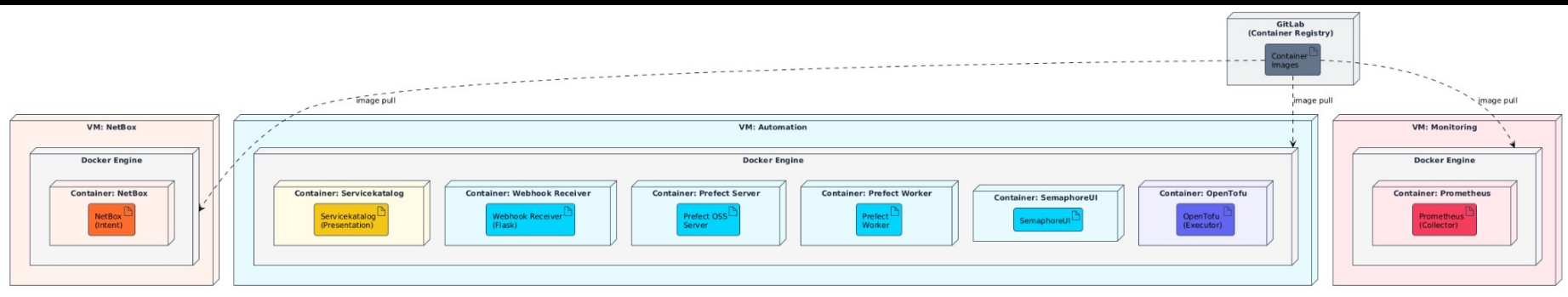
# Building Blocks



# Runtime View



# Deployment



# Summary



Documentation is a design  
tool, not a bureaucratic  
artifact.

# Takeaways

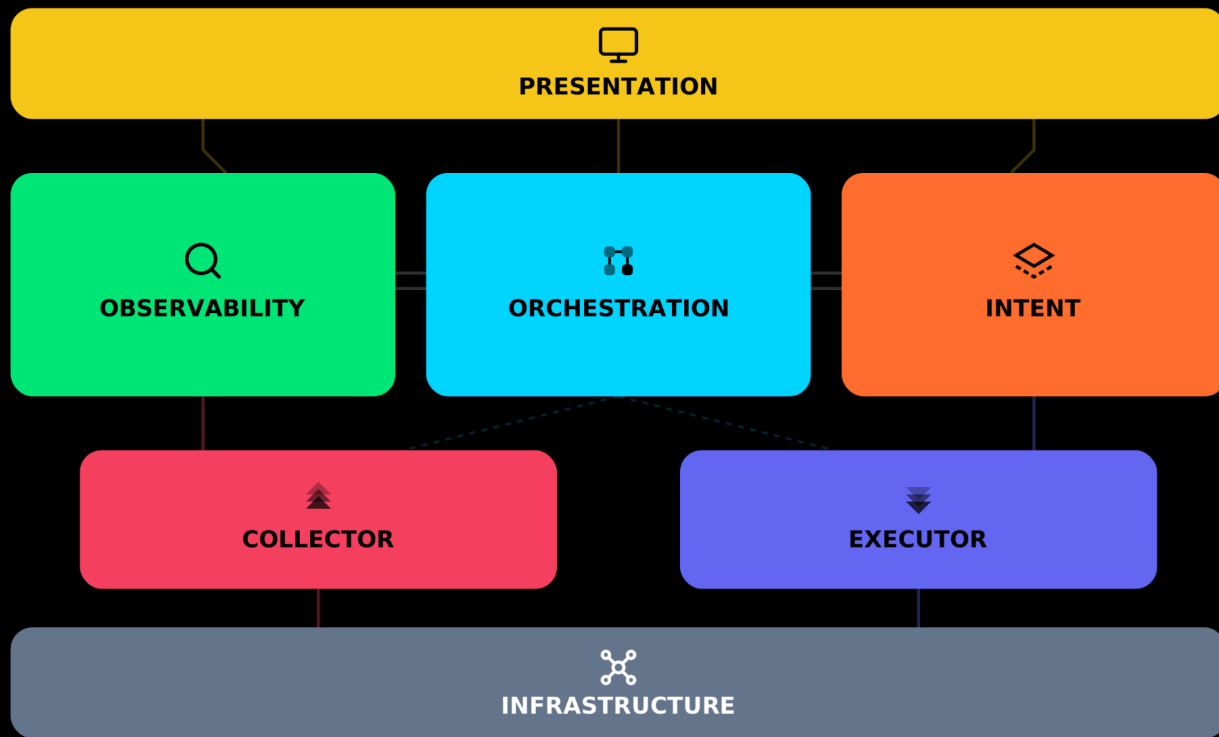
- Automation platforms are ecosystems – design them like networks.
- Document everything for compliance.
- Use AI-readable tooling.
- Apply NAF to complement existing standards.



Questions?

# Appendix

# Personal Gear - NAF Framework



Scripting alone doesn't scale

## Personal Gear - ADRs

### Architectural Design Records (ADR)

- IEC 42010:2022, around since late '90s
- Context about the WHY of a design.
- Various templates out there. Well known is Michael Nygard's version with structure:

TITLE | STATUS | CONTEXT | DECISION | CONSEQUENCES

# Personal Gear - C4

## C4 Model

- Created by Simon Brown
- Focus on visualising software architecture at different abstraction levels
- The "4 Cs"  
CONTEXT | CONTAINERS | COMPONENTS | CODE



# Template Slides













